

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 7_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is $3 + 4 = 7$.

Input Format

The first line of input contains an integer `num_cities`, representing the number of cities (numbered 0 to `num_cities - 1`).

The second line contains an integer `num_segments`, representing the number of undirected segments (edges) between cities.

The next `num_segments` lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

Output Format

The first line of output prints an integer representing the minimum cost to connect all cities.

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 4

1 2 5

0 2 3

Output: 7

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int u, v, w;  
} Edge;
```

```
int parent[20], rankArr[20];
```

```
int find(int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

```
void unionSet(int a, int b) {  
    int pa = find(a);  
    int pb = find(b);
```

```
    if (pa != pb) {  
        if (rankArr[pa] < rankArr[pb])  
            parent[pa] = pb;  
        else if (rankArr[pa] > rankArr[pb])  
            parent[pb] = pa;  
        else {
```

```

    parent[pb] = pa;
    rankArr[pb]++;
  }
}

```

```

int cmp(const void *a, const void *b) {
    return ((Edge *)a)->w - ((Edge *)b)->w;
}

```

```

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    Edge edges[100];

    for (int i = 0; i < m; i++)
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);

    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }

    qsort(edges, m, sizeof(Edge), cmp);

    int count = 0, cost = 0;

    for (int i = 0; i < m; i++) {
        if (find(edges[i].u) != find(edges[i].v)) {
            unionSet(edges[i].u, edges[i].v);
            cost += edges[i].w;
            count++;
        }
    }

    if (n == 1) {
        printf("0");
    } else if (count == n - 1) {
        printf("%d", cost);
    } else {

```

```
        printf("It is not possible to connect all cities");  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Maya, a project planner, must connect multiple office branches using network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

Input Format

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

Output Format

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3 5

1 2 2

2 3 3

3 1 4

Output: Minimum Cost Spanning Tree within the budget: 5

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int u, v, w;  
} Edge;
```

```
int parent[105], rankArr[105];
```

```
int find(int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

```
void unionSet(int a, int b) {  
    int pa = find(a);  
    int pb = find(b);  
    if (pa != pb) {  
        if (rankArr[pa] < rankArr[pb])  
            parent[pa] = pb;  
        else if (rankArr[pa] > rankArr[pb])  
            parent[pb] = pa;  
        else {  
            parent[pb] = pa;  
            rankArr[pa]++;  
        }  
    }  
}
```

```
}
```

```
int cmp(const void *a, const void *b) {  
    return ((Edge *)a)->w - ((Edge *)b)->w;  
}
```

```
int main() {  
    int V, E;  
    long long budget;  
    scanf("%d %d %lld", &V, &E, &budget);
```

```
    Edge edges[1005];
```

```
    for (int i = 0; i < E; i++)  
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
```

```
    for (int i = 1; i <= V; i++) {  
        parent[i] = i;  
        rankArr[i] = 0;  
    }
```

```
    qsort(edges, E, sizeof(Edge), cmp);
```

```
    long long cost = 0;  
    int count = 0;
```

```
    for (int i = 0; i < E; i++) {  
        if (find(edges[i].u) != find(edges[i].v)) {  
            unionSet(edges[i].u, edges[i].v);  
            cost += edges[i].w;  
            count++;  
        }  
    }
```

```
    if (count == V - 1 && cost <= budget)  
        printf("Minimum Cost Spanning Tree within the budget: %lld", cost);  
    else  
        printf("A valid MST within the budget is not possible.");
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

Input Format

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between those two locations.

Output Format

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

0 5

5 0

Output: Location Distance

0 -> 1 5 km

Minimum total distance: 5 km

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define MAX 10

void findMST(int n, int graph[MAX][MAX]) {
    int parent[MAX];
    int key[MAX];
    bool mstSet[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = false;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n - 1; count++) {
        int min = INT_MAX, u;

        for (int v = 0; v < n; v++) {
            if (mstSet[v] == false && key[v] < min) {
                min = key[v];
                u = v;
            }
        }
    }
}
```

```

        mstSet[u] = true;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printf("Location\tDistance\n");
    int totalDistance = 0;
    for (int i = 1; i < n; i++) {
        printf("%d -> %d\t\t\t%d km\n", parent[i], i, graph[i][parent[i]]);
        totalDistance += graph[i][parent[i]];
    }
    printf("Minimum total distance: %d km\n", totalDistance);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int graph[MAX][MAX];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    findMST(n, graph);

    return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

Input Format

The first line contains an integer n , representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

Output Format

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

'<symbol>: <binary_code>'

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

a 1
b 2

Output: a: 0
b: 1

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Node {
    char data;
    unsigned freq;
    struct Node *left, *right;
};
```

```
struct MinHeap {
    unsigned size;
    unsigned capacity;
    struct Node** array;
};
```

```
struct Node* newNode(char data, unsigned freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}
```

```
struct MinHeap* createMinHeap(unsigned capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    minHeap->array = (struct Node**)malloc(minHeap->capacity * sizeof(struct Node*));
    return minHeap;
}
```

```
void swapNode(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
```

```

    *b = t;
}

void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->freq < minHeap->array[smallest]->freq)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->freq < minHeap->array[smallest]->freq)
        smallest = right;

    if (smallest != idx) {
        swapNode(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}

```

```

struct Node* extractMin(struct MinHeap* minHeap) {
    struct Node* temp = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    --minHeap->size;
    minHeapify(minHeap, 0);
    return temp;
}

```

```

void insertMinHeap(struct MinHeap* minHeap, struct Node* minHeapNode) {
    ++minHeap->size;
    int i = minHeap->size - 1;
    while (i && minHeapNode->freq < minHeap->array[(i - 1) / 2]->freq) {
        minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = minHeapNode;
}

```

```

void buildMinHeap(struct MinHeap* minHeap) {
    int n = minHeap->size - 1;

```

```

int i;
for (i = (n - 1) / 2; i >= 0; --i)
    minHeapify(minHeap, i);
}

```

```

int isLeaf(struct Node* root) {
    return !(root->left) && !(root->right);
}

```

```

struct MinHeap* createAndBuildMinHeap(char data[], int freq[], int size) {
    struct MinHeap* minHeap = createMinHeap(size);
    for (int i = 0; i < size; ++i)
        minHeap->array[i] = newNode(data[i], freq[i]);
    minHeap->size = size;
    buildMinHeap(minHeap);
    return minHeap;
}

```

```

struct Node* buildHuffmanTree(char data[], int freq[], int size) {
    struct Node *left, *right, *top;
    struct MinHeap* minHeap = createAndBuildMinHeap(data, freq, size);

    while (minHeap->size != 1) {
        left = extractMin(minHeap);
        right = extractMin(minHeap);
        top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        insertMinHeap(minHeap, top);
    }
    return extractMin(minHeap);
}

```

```

void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
}

```

```

    if (isLeaf(root)) {
        printf("%c: ", root->data);
        for (int i = 0; i < top; ++i)
            printf("%d", arr[i]);
        printf("\n");
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    char data[256];
    int freq[256];

    for (int i = 0; i < n; i++) {
        scanf(" %c %d", &data[i], &freq[i]);
    }

    if (n == 1) {
        printf("%c: 0\n", data[0]);
        return 0;
    }

    struct Node* root = buildHuffmanTree(data, freq, n);
    int arr[256], top = 0;
    printCodes(root, arr, top);

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*". The left edge (0) leads to the symbol "A". The right edge (1) leads to the symbol "B".

This means:

"0" A "1" B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

Input Format

The first line contains an integer n, representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be decoded

Output Format

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 11100011100

Output: BBBAABBBA

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```



```
int main() {
    char input[200];
    char encoded[200];

    if (scanf("%s", input) != 1) return 0;

    if (input[0] == '0' || input[0] == '1') {
        strcpy(encoded, input);
    } else {
        int n = atoi(input);
        char ch, code[100];
        for (int i = 0; i < n; i++)
            scanf(" %c %s", &ch, code);
        scanf("%s", encoded);
    }

    for (int i = 0; encoded[i]; i++) {
        if (encoded[i] == '0')
            printf("A");
        else
            printf("B");
    }

    return 0;
}
```

Status : Correct

Marks : 10/10